

AUDITORIA TÉCNICA INDEPENDENTE — RADAR TASTYTRADE PRO IA

Ciclo 5 · Auditoria cega + reconciliação contra Ciclo 1 e Ciclo 2

Escopo: C:\Projetos AntigraVity\RADAR-TASTYTRADE **Data:** 08/09/2026

Auditor: sessão independente, sem participação em correções anteriores deste projeto **Método:** varredura estática completa de src/lib/domain, src/lib/services, src/lib/security, src/app/api, src/components; execução de tsc --noEmit, eslint e vitest run; reexecução manual e independente dos motores BSM e GEX com casos de teste próprios; reconciliação contra os laudos dos Ciclos 1 e 2 recuperados do histórico Git

0. NOTA SOBRE O ESCOPO DESTES CICLO — LEIA ANTES DO RESTANTE

O prompt que originou esta auditoria presumia a existência de quatro laudos anteriores (Ciclo 1 a Ciclo 4) na raiz do projeto e de uma seção “Mapa de Correções Alegadas” anexada ao próprio prompt. Nenhuma das duas premissas se confirmou:

- **Só existem, de fato, dois laudos anteriores:**
AUDITORIA_RADAR_TASTYTRADE.md (Ciclo 1) e
AUDITORIA_RADAR_02_REAUDITORIA.md (Ciclo 2). Nunca existiu Ciclo 3 ou Ciclo 4 versionado neste repositório — confirmado por busca em todo o histórico Git (git log --all --diff-filter=A --name-only).
- Esses dois laudos, mais o guia PROMPT_AUDITORIA.md, **foram apagados do disco sem commit** (git status os mostra como D, deletados na working tree, nunca removidos via commit). Alguém — pessoa ou sessão anterior — os removeu manualmente depois do último commit (56a9049), e essa remoção não está registrada no histórico. Isso por si só é um achado (ver C5-13).
- O código-fonte atual contém **dezenas de comentários** citando um “Ciclo 4” e achados específicos dele (ex.: “Achado D-01 do laudo Ciclo 4”, “Nível 1 Parte 3 da remediação, Ciclo 4”, “decidido com o usuário em 2026-09-08”). Isso indica que um terceiro ciclo de auditoria/remediação provavelmente ocorreu — mas nenhum laudo desse ciclo sobrevive em disco ou em commit para eu ler. Não tenho como reconstituir o que ele efetivamente encontrou ou reivindicou, além do que os próprios comentários no código revelam.

Diante disso, e por instrução explícita do usuário, esta auditoria segue o processo de duas fases normalmente, mas a **Fase 2 reconcilia contra o Ciclo 1 e o Ciclo 2** (os únicos laudos recuperáveis), tratando o hiato documental do “Ciclo 3/4” como um achado de governança em si — não como uma lacuna a preencher por suposição.

1. PARECER EXECUTIVO

**VEREDITO: REPROVADO PARA ALOCAÇÃO DE CAPITAL
REAL. Integridade de dados: 5,3 / 10** (Ciclo 1: 2,8 → Ciclo 2:
4,2 → agora: 5,3)

Houve progresso genuíno e verificável entre o Ciclo 2 e hoje — mais do que entre o Ciclo 1 e o Ciclo 2. Refiz as contas de próprio punho e confirmo:

- O motor Black-Scholes-Merton (`bsm-pricer.ts`) está matematicamente correto — validei preço, delta, gama e vega contra cálculo independente em Python, incluindo a checagem de paridade put-call, com diferença de arredondamento apenas na última casa.
- O motor GEX (`gex-engine.ts`) está correto na convenção Dollar $\Gamma = \Gamma \times \text{OI} \times S^2 \times 100 \times 0,01 / 1.000.000$, validado numericamente por mim.
- Os prêmios de opções, antes um percentual fixo do spot (achados A-04/N-02/N-03 dos ciclos anteriores), **agora vêm do motor BSM real**, alimentado por IV e DTE de fato — verifiquei o código linha a linha e o crédito líquido agora varia genuinamente com a volatilidade informada.
- O breakeven do Bull Put Spread não fabrica mais um teto superior inexistente ($\text{spot} \times 1,05$) — achado N-07 do Ciclo 2, confirmado corrigido.
- O override hardcoded da VALE3 (A-06) segue removido nos três arquivos.
- RSI de Wilder e MACD de Gerald Appel agora são implementados com a fórmula canônica correta (`calculateWilderRSI`, `calculateCanonicalMACD`) — achado N-06 do Ciclo 2, corrigido.
- A rota de segurança (`api-guard.ts`) foi reescrita: validação por `Sec-Fetch-Site` (não mais um header que o cliente controla), sessão via `cookie httpOnly`, teto global — corrige a falha mais grave do achado N-04.
- `docs/fontes/` agora existe com payloads reais salvos (`option-chains-nested.sample.json`, `market-metrics.sample.json`) — o portão G3 (Prova de Fonte), citado no Ciclo 2 como “o portão que foi pulado”, parece ter sido finalmente executado nesse ciclo intermediário sem laudo.
- ESLint agora tem duas regras ativas (`Math.random()` proibido, `catch` vazio proibido) e está no CI — achado N-09 do Ciclo 2, parcialmente corrigido.

Isto não é maquiagem. É trabalho de engenharia real, concentrado exatamente nas fórmulas que os dois laudos anteriores apontaram como erradas.

Mas a causa-raiz identificada desde o Ciclo 1 continua de pé, e ela é suficiente, sozinha, para reprovar o sistema:

A tela principal do produto (Cotação/Fundamentos, `QuoteView.tsx`) continua sendo alimentada por dois catálogos inteiramente estáticos — `US_STOCKS_DATASET` (64 tickers) e `SP500_DATASET` (48 tickers) — e rotula esse dado congelado como “SPOT: REAL (TASTYTRADE)” de forma incondicional, além de exibir texto fixo (“OPÇÕES: NET GEX POSITIVO”, “MACRO: FED 4,50%”) que nunca é recalculado, para qualquer ativo, em qualquer dia.

Isso é, em espécie, o mesmo achado A-07/N-01 dos ciclos anteriores — só que agora não está mais no motor de GEX (esse foi corrigido: o `getGexAnalysis()` problemático foi deletado), está na tela de abertura do produto, no primeiro número que o usuário vê. E há uma reincidência agravante: o sistema de proveniência de dado que os dois laudos pediram (`badges MEDIDO/DERIVADO/ESTIMADO/SIMULADO`) **foi construído** (`src/lib/types/provenance.ts` existe, com a lógica de contágio correta) **e não é usado em lugar nenhum do código** — zero import em qualquer componente. A infraestrutura mais recomendada em dois laudos consecutivos está pronta e desconectada.

Achei também uma regressão pontual e concreta: o padrão de constantes fabricadas de fundamentos (`currentRatio`, `ebitdaMargin`, `priceToBook` como números fixos) que o próprio código afirma ter sido

eliminado de QuoteView.tsx no “Ciclo 4” **continua presente, palavra por palavra, em ai-consultant.ts** — a correção foi feita em um consumidor do motor de fundamentos e não no outro.

2. FASE 1 — ACHADOS DA VARREDURA CEGA

Numerados C5-XX para não colidir com a numeração dos laudos anteriores. Cada um foi levantado antes de eu ler qualquer laudo antigo.

CRÍTICOS

C5-01 — Os dois catálogos que alimentam a tela principal são, em essência, constantes hardcoded por ticker `src/lib/domain/us-market-data.ts:41+` (`US_STOCKS_DATASET`, 64 tickers) e `src/lib/domain/sp500-dataset.ts:28+` (`SP500_DATASET`, 48 tickers). Cada linha é um objeto literal completo — `spot`, `change`, `roe`, `stop`, `alvo1`, `alvo2`, `netGex`, `putWall`, `callWall`, `iv30` etc. — escrito à mão, sem nenhum campo de timestamp ou proveniência. É exatamente o padrão que a REGRA 00 proíbe no invariante 7 (“constante hardcoded por entidade específica”), só que como array de objetos em vez de `if (symbol === 'X')`. Nenhum refresh ao vivo existe para `spot`, `stop`, `alvo1`, `alvo2`, `netGex`, `zeroGammaFlip`, `putWall`, `callWall` em nenhum dos dois catálogos.

C5-02 — “SPOT: REAL (TASTYTRADE)” é um rótulo incondicional sobre dado estático

`src/components/quote/QuoteView.tsx:317`. O texto é renderizado sempre, sem checagem de fonte ao vivo, para `currentStock.spot`, que vem 100% de `US_STOCKS_DATASET` (achado C5-01). Não existe, em nenhum ponto de `QuoteView.tsx`, uma chamada a `/api/market/metrics` ou qualquer outro endpoint para atualizar o `spot`. Isso viola diretamente a proibição da REGRA 00: “*Não rotule dado como ‘ao vivo’, ‘tempo real’, ‘oficial’ ou ‘idêntico à plataforma’ se qualquer campo da tela for estático.*”

C5-03 — Texto fixo apresentado como fato calculado por ativo

`src/components/quote/QuoteView.tsx:321,323`:

```
<span>3. OPÇÕES: NET GEX POSITIVO</span>
...
<span>4. MACRO: FED 4.50%</span>
```

Não há interpolação (`{}`) — é string JSX literal. Todo ticker, em qualquer situação de GEX real (que pode ser negativo) ou taxa Fed real (que muda ao longo do tempo), mostra exatamente esse texto. Isto não é dado estático rotulado incorretamente — é uma afirmação fabricada sem relação alguma com o ativo em tela.

C5-04 — Regressão: constantes fabricadas de fundamentos reaparecem em ai-consultant.ts `src/lib/domain/ai-consultant.ts:74-76`:

```
currentRatio: 1.45,
ebitdaMargin: 0.28,
priceToBook: Number((stock.peRatio / 18).toFixed(2)),
```

São os mesmos três valores/heurística que o comentário em `QuoteView.tsx:113-119` descreve como removidos (“Antes esta tela tinha sua PRÓPRIA cópia divergente das constantes fabricadas (1.45/0.28, e um P/VP estimado por `peRatio/18` [...]). Agora null nos dois lugares”). A correção foi aplicada em `QuoteView.tsx` e não em `ai-consultant.ts`, que consome o mesmo `fundamentalsEngine.evaluate()`. O “Consultor Quantitativo IA” — que responde perguntas de usuário sobre a saúde financeira do ativo — ainda fabrica esses três números.

ALTOS

C5-05 — Símbolo de opção (OCC) com vencimento fixo

fabricado `src/lib/domain/gex-engine.ts:141,153:`

```
symbol: `.${symbol.toUpperCase()}260918C${strikeStr}`,
```

A data “260918” (18/set/2026) está hardcoded no símbolo de toda call/put wall gerada, independente do vencimento real da opção em questão. É identidade de instrumento fabricada.

C5-06 — Fallbacks por constante mágica dentro do motor GEX

`src/lib/domain/gex-engine.ts:144,156: iv: s.callIv || 35, delta: s.callDelta || 0.5 (e -0.5 para put). gex-engine.ts:161-162:`

```
const topCallWall = topCallStrikes[0]?.strike || spotPrice * 1.05;  
const topPutWall = topPutStrikes[0]?.strike || spotPrice * 0.95;
```

Este último é, literalmente, o exemplo que a própria REGRA 00 cita como proibido (“nunca spot * 1.05”), presente ainda hoje no motor central de GEX.

C5-07 — O sistema de proveniência pedido em dois laudos

existe e está órfão `src/lib/types/provenance.ts` define

`ProvenanceBadge` (MEDIDO/DERIVADO/ESTIMADO/SIMULADO) e

`combineProvenance()` com a regra de contágio correta (simulado contamina a cadeia). Busquei por qualquer import desse módulo em todo `src/`: **zero ocorrências fora do próprio arquivo.** A

recomendação de maior impacto e menor custo do Ciclo 1, reiterada no Ciclo 2, foi implementada como tipo e nunca conectada a um componente.

C5-08 — README declara uma garantia que o próprio sistema

viola `README.md:8`: “nenhum número sintético é mascarado como dado de mercado em tempo real”. À luz de C5-02/C5-03, essa frase está factualmente incorreta sobre o estado atual do sistema — mesma classe dos achados A-20 (Ciclo 1) e N-10 (Ciclo 2), que também apontaram o README fazendo promessas que o código não cumpria.

MÉDIOS

C5-09 — Taxa livre de risco fixa em 4,5% ao ano `bsm-pricer.ts:60`

(`riskFreeRate = 0.045`) e `volatility-engine.ts` (`rRate = 0.045`, comentado como “SOFR / Fed Funds”). Nunca atualizada por fonte real. Severidade média porque a sensibilidade do preço BSM a `r` é baixa em opções de curto prazo (30-35 DTE, como usado aqui) — mas ainda é um parâmetro de precificação tratado como constante, não como dado.

C5-10 — Teste de integração sem mock, não determinístico

`src/lib/services/tastytrade-market.service.test.ts:5-18` chama `getMarketMetrics()` real, que faz uma requisição de rede de verdade à API da Tastytrade. Nesta auditoria, falhou por falta de rede/credencial no ambiente (fetch failed) — o mesmo aconteceria em qualquer CI sem as credenciais OAuth configuradas como secret. Teste não hermético; não é “decorativo” (ele afirma propriedades reais), mas é frágil e ambiente-dependente.

C5-11 — Rate limit em memória não sobrevive a múltiplas

instâncias `src/lib/security/api-guard.ts` documenta honestamente essa limitação no próprio comentário do arquivo. Sob escala horizontal (múltiplas instâncias serverless), o teto efetivo é $N \times$ o configurado, já que cada instância mantém seu próprio Map. Não é uma alegação falsa (está documentado), mas segue sem mitigação — só é mitigável com um store externo (Redis/Upstash).

C5-12 — Teto mágico `Math.min(debtToEbitda, 1.2)` sem fonte

declarada Presente tanto em `QuoteView.tsx:112` quanto em `ai-consultant.ts:70` (duplicado). Não há comentário explicando a origem

do valor 1,2 nem por que é aplicado só a `financialDebtToEbitda` e não a `debtToEbitda`.

BAIXOS

C5-13 — Trilha de auditoria apagada do disco sem commit Ver Seção 0. `AUDITORIA_RADAR_TASTYTRADE.md/.pdf`, `AUDITORIA_RADAR_02_REAUDITORIA.md/.pdf` e `PROMPT_AUDITORIA.md` existem no histórico Git mas foram removidos da `working tree` sem commit de remoção. Rastreabilidade do próprio processo de auditoria comprometida.

C5-14 — Capacidade real implementada e não conectada `GET /api/market/fundamentals (src/app/api/market/fundamentals/route.ts)` usa `brapiService`, que faz chamadas reais à API BRAPI e propaga `null` honestamente quando o dado não existe (nenhuma fabricação encontrada nesse serviço). Busquei por qualquer chamada a essa rota em `src/components/` e `src/app/`: nenhuma. É capacidade real ociosa — o inverso do problema usual, mas ainda um gap de execução: dado real disponível, não usado onde poderia substituir dado estático.

C5-15 — Zero Gamma Flip por ponto médio simples, não por interpolação proporcional `gex-engine.ts:168-176` toma o ponto médio aritmético entre os dois strikes adjacentes onde o Net GEX muda de sinal, em vez de interpolar proporcionalmente ao valor de `netGex` em cada strike. É uma aproximação razoável, mas não documentada como tal.

C5-16 — Barreira automática cobre 2 dos 8 invariantes da REGRA 00 `.eslintrc.json` só bloqueia `Math.random()` e `catch` vazio. Os padrões que causaram C5-01, C5-02, C5-03, C5-04 e C5-06 (constante mágica, `hardcode` por entidade, rótulo de proveniência ausente/ incorreto, `fallback spot × constante`) não têm nenhuma barreira mecânica — dependem de leitura humana do laudo, exatamente o diagnóstico estrutural que o próprio Ciclo 2 já havia feito (N-09) e que segue parcialmente verdadeiro.

PROVA DE EXECUÇÃO (regra de evidência)

- `npx tsc --noEmit -p tsconfig.json` — **limpo, zero erros**.
- `npx eslint src` — **limpo, zero violações** (esperado: cobre só 2 invariantes, ver C5-16).
- `npx vitest run` — **41 de 42 testes passando**. A falha é C5-10 (dependência de rede real), não um bug de lógica.
- Motor BSM (`calculateBsm`) executado com `spot=142,50`, `strike=145`, `T=30/365`, `vol=30%`, `r=4,5%`: `CALL preço=4,00 delta=0,45 gamma=0,0323 vega=0,162`; `PUT preço=5,97 delta=-0,55`. Conferido contra implementação independente em Python (`normal_cdf` via `math.erf`): `CALL preço=4,0015 delta=0,4537 gamma=0,032332 vega=0,1619`; `PUT preço=5,9662 delta=-0,5463`. Diferença apenas de arredondamento. **Paridade put-call conferida exatamente** (diferença 0,00000000).
- Motor GEX (`calculateGex`) executado com `gamma=0,03`, `OI=5000`, `spot=142,50`, `contractSize=100`: `callGex=3,0459 $M`. Conferido contra cálculo manual: $0,03 \times 5000 \times 142,5^2 \times 100 \times 0,01 / 1.000.000 = 3,0459 \M . **Exato**.

3. FASE 2 — RECONCILIAÇÃO

3.1 Placar consolidado: Ciclo 1 → Ciclo 2 → Ciclo 5 (agora)

#	Achado original (Ciclo 1)	Status Ciclo 2	Status agora (Ciclo 5)
---	------------------------------	-------------------	---------------------------

A-01	Candles/indicadores via Math.random()	● PARCIAL	● PARCIAL (mantido)	Série ruído RSI de corret calcul HISTO Candle
A-02	Cadeia de opções sintética / N-01 (rotulada tastytrade-live falsamente)	● PIOROU (N-01, novo)	● RESOLVIDO	getGex deleta (confi ausên do N-0 Unifie corret NÃO I
A-03	Spot hardcoded em 3 datasets divergentes	● INTACTO	● PARCIAL	Uma c remov catálo SP500 lugar
A-04	Prêmios/crédito fabricados (% fixo do spot)	● PARCIAL (regra 1/3 ok, prêmio ainda %spot)	● RESOLVIDO EM SUBSTÂNCIA	Prêmi com I mater input. estáti herda com fi
A-05	Rotas sem auth/rate limit / N-04 (rate limit burlável por header)	● PARCIAL	● RESOLVIDO EM GRANDE PARTE	api-gu Fetch- sessã aplica auten memó
A-06	Override hardcoded VALE3	● RESOLVIDO	● CONFIRMADO RESOLVIDO	Nenhu encon brapi.
A-07	Badge "live" sobre dado estático / N-01	● PARCIAL	● PARCIAL — mesmo padrão, componente diferente	Unifie Volati condic live') 03, qu achad olhad
A-08	Enriquecimento parcial live+estático (spot/walls congelados, só IV atualiza)	● INTACTO	● INTACTO	Volati só ivr/iv são sc netGex nunca
A-09	Heurística de unidade >1 ? % : fração	● INTACTO	● RESOLVIDO	fundam contra (toPer corree
A-10	Breakeven por percentual fixo do spot / N-07 (novos casos: calendar, bull put)	● RESOLVIDO (IC/Bull Put) / ● N-07 novo	● RESOLVIDO INTEGRALMENTE	volati tipos Calen strike fabric do cal
A-11	POP fixo 72%/48%	● COSMÉTICO (variava 70-74)	NÃO RE-VERIFICADO NESTA SESSÃO	Não r como não co
A-12	Max Loss ignora segunda asa	● RESOLVIDO	● CONFIRMADO	maxLos max(pu

			RESOLVIDO	prepa
A-13	Testes tautológicos / N-05 (mesmo vício, formato novo)	● PERSISTE	NÃO RE-VERIFICADO EM DETALHE	Vi que toBe(1 regist a linha
A-14	Fallbacks silenciosos mascararam falha de API	● INTACTO	● PARCIAL	tastyt regist inven (come código leitura menor
A-15	Símbolo OCC incompatível entre módulos	● RESOLVIDO	● PARCIAL	O forr vencer fixo/fa difere
A-16	Leitor manual de .env.local	● INTACTO	● INTACTO	tastyt parsei só log
A-17	ANTHROPIC_API_KEY órfã	● INTACTO	NÃO RE-VERIFICADO	Não in do esc
A-18	Token OAuth em arquivo plano	● INTACTO	● INTACTO (mitigação parcial)	tasty_ plano .gitig versio
A-19	Cache sem evicção	● RESOLVIDO	● CONFIRMADO RESOLVIDO	tastyt evicçã
A-20 / N-10	README com alegações falsas	● PIOROU	● PARCIAL — nova alegação falsa, tipo diferente	“100% apare README sintét luz de
A-21 / N-09	Sem CI / sem lint / sem barreira	● PARCIAL	● PARCIAL — melhor, ainda incompleto	CI cor confir agora cobre (C5-10
A-22 / N-11	Working tree divergente do repositório	● PIOROU	NÃO MENSURADO DA MESMA FORMA	Há tra també arquiv correç recorri
A-23	.mp4 duplicado 43MB	● INTACTO	NÃO RE-VERIFICADO	Fora c

Resumo do placar: de 23 achados do Ciclo 1, considerando a evolução até agora: **9 resolvidos ou confirmados resolvidos, 7 parciais, 3 intactos, 4 não re-verificados nesta sessão** (declarado explicitamente, não estimado).

3.2 Achados do Ciclo 1/2 que eu NÃO encontrei sozinho na Fase 1 — e por quê

- **A-01 (candles aleatórios), A-06 (VALE3), A-09 (heurística de unidade), A-10/N-07 (breakeven %spot), A-12 (max loss), A-19 (cache sem evicção):** não apareceram na minha varredura cega porque **foram genuinamente corrigidos** — não porque eu tenha passado por cima. Confirmei cada um lendo o código atual antes de olhar os laudos antigos.
- **A-02/N-01 (GEX rotulado falsamente como live):** não apareceu porque o método específico que causava isso (getGexAnalysis()) foi deletado. O padrão retornou, mas em outro arquivo (C5-02/C5-03)

— por isso ele não bateu com a busca textual pelo achado antigo.

3.3 Achados novos desta sessão que os laudos anteriores não tinham

C5-04 (regressão em `ai-consultant.ts`), C5-05 (vencimento fixo no símbolo OCC), C5-07 (`provenance.ts` órfão), C5-09 (taxa livre de risco fixa), C5-12 (teto mágico 1,2), C5-13 (trilha de auditoria apagada), C5-14 (rota BRAPI real não conectada), C5-15 (interpolação simplificada do Zero Gamma Flip). Nenhum destes constava nos laudos de Ciclo 1 ou 2.

3.4 Achados introduzidos pela correção anterior (regressões)

- **C5-04** é, em espécie, uma regressão: a correção do achado “constantes fabricadas de fundamentos” foi aplicada em `QuoteView.tsx` e não em `ai-consultant.ts`, que consome o mesmo motor. O código teve, num certo momento, esse defeito reintroduzido/deixado para trás num segundo consumidor — o mesmo padrão de “correção espalhada e não sincronizada” que o próprio Ciclo 1 já havia apontado no caso VALE3 (A-06, item 3: “a lógica está espalhada em três arquivos”).
- **C5-02/C5-03** não são tecnicamente uma regressão de um achado específico anterior, mas são a mesma classe de defeito (A-07/N-01: rótulo de dado real sobre dado estático) reaparecendo num componente que os ciclos anteriores não haviam examinado em detalhe.

4. O QUE MELHOROU DE VERDADE — SEM RESSALVA

1. **Motor de precificação de opções agora é real.** BSM correto, alimentado por IV/DTE reais, não mais percentual fixo do spot. Isto era, nas palavras do Ciclo 1, “o achado de maior potencial de dano financeiro direto” — e foi corrigido em substância, não em aparência.
2. **GEX sintético agora se declara sintético.** O caminho que rotulava dado fabricado como `tastytrade-live` foi removido; o que restou se identifica corretamente como “MODELO CALIBRADO”.
3. **Segurança de API:** validação de origem não mais burlável por header de cliente, sessão via cookie `httpOnly`, rate limit em camadas, aplicado consistentemente nas 5 rotas.
4. **RSI e MACD com fórmula canônica correta**, não mais aproximações rotuladas com nome errado.
5. **Prova de Fonte executada:** `docs/fontes/` existe com payloads reais — o portão que o Ciclo 2 apontou como pulado parece ter sido finalmente cumprido nalgum momento entre Ciclo 2 e agora.
6. **CI com lint + typecheck + testes + build**, e ESLint com duas regras REGRA-00-específicas ativas.
7. **VALE3, unidade de percentual, cache sem evicção, max loss assimétrico:** todos confirmados resolvidos por leitura direta do código atual, não por comentário.

5. RISCO — ONDE ASSUMIR, ONDE MITIGAR

Risco	Decisão	Racional
		Reprodutível, rotulada,

Série de candles sintética, rotulada "SÉRIE HISTÓRICA MODELADA"	ASSUMIR	indicadores agora corretos sobre a série q existe. Aceitáve até haver OHLCV real. É a causa-raiz (três ciclos de auditoria. Sem isso resolvido, nada mais no sistema import a base de todo cálculo é fabricada. Rótulo falso sobre dado estático é exatamente o que a REGRA 0 chama de "pio que dado 100% falso". Correçã é de baixo cust condicionar o texto ao source real ou removê lo.
US_STOCKS_DATASET/SP500_DATASET estáticos alimentando spot/stop/alvos	MITIGAR — bloqueante	Não há custo d engenharia relevante em interpolar ou remover; é a linha mais barata de corrigir em tod este laudo.
"SPOT: REAL (TASTYTRADE)" incondicional em QuoteView	MITIGAR — bloqueante, correção trivial	Repetir a correção já feit em QuoteView.t (passar null e deixar o motor tratar).
"NET GEX POSITIVO"/"FED 4,50%" como texto fixo	MITIGAR — bloqueante, correção trivial	Não é bloqueante por si, mas é a peç que resolveria estruturalment C5-02/C5-03/A-07 se fosse de fato conectada UI. Prioridade alta por ser a correção de maior alavancagem.
Regressão de fundamentos fabricados em ai-consultant.ts	MITIGAR — bloqueante, correção trivial	Documentado honestamente, não é alegação falsa. Mitigar quando o tráfe justificar múltiplas instâncias.
provenance.ts órfão	MITIGAR	Baixo impacto precificação de
Rate limit em memória não distribuído	ASSUMIR temporariamente	

Taxa livre de risco fixa em 4,5%	ASSUMIR	opções de curto prazo; mitigar quando houver fonte de Treasury/SOFR disponível. Adicionar mock ou marcar com teste de integração separado do vitest run padrão do CI.
Teste de integração sem mock (C5-10)	MITIGAR — barato	É o motivo estrutural de C01 a C5-06 terem sobrevivido a três ciclos. Regra customizada contra <code> <número> e * 1.0\d</code> em <code>domain//service</code> é viável e barato
Barreira ESLint cobrindo só 2 de 8 invariantes	MITIGAR	

6. RECOMENDAÇÃO FINAL

O sistema não deve ser usado para alocação de capital real na configuração atual. Mas a natureza do bloqueio mudou desde o Ciclo 1: não é mais “a matemática financeira está errada” — está, em sua maior parte, correta e verificada por execução independente nesta auditoria. O bloqueio é: **a tela que o usuário abre primeiro (Cotação) ainda apresenta dado estático como se fosse cotação real da Tastytrade, sem nenhuma condição, e complementa isso com duas linhas de texto fixo que fingem ser conclusões calculadas.**

Isto é corrigível rápido — mais rápido que qualquer fase dos planos de remediação anteriores — porque o trabalho pesado (motor BSM real, motor GEX real, tipo de proveniência já definido) já está feito e ocioso. Estimativa: **3 a 5 dias** de trabalho focado, não semanas:

1. **Hoje:** remover ou condicionar "SPOT: REAL (TASTYTRADE)", "NET GEX POSITIVO" e "FED 4,50%" em `QuoteView.tsx:317-323`. É a correção de menor custo e maior redução de risco reputacional/regulatório deste laudo inteiro.
2. **Esta semana:** conectar `src/lib/types/provenance.ts` a `QuoteView.tsx`, `VolatilityAnalystView.tsx` e `UnifiedGexBarreirasView.tsx` — um badge por campo, não por tela inteira, substituindo os rótulos manuais e inconsistentes que existem hoje em cada componente.
3. **Esta semana:** replicar em `ai-consultant.ts` a correção já feita em `QuoteView.tsx` para `currentRatio/ebitdaMargin/priceToBook` (C5-04).
4. **Próxima:** decidir e executar a consolidação de `US_STOCKS_DATASET/SP500_DATASET` num catálogo único com fonte real por campo — ou, no mínimo, exibir explicitamente a data de congelamento de cada literal (“dado de 08/09/2026, não atualizado”) em vez do rótulo “REAL”.
5. **Em paralelo, barato:** estender a regra de ESLint para capturar o padrão `|| <literal numérico> e <var> * 1.0\d` dentro de `src/lib/domain/` e `src/lib/services/` — isso teria pego C5-06 automaticamente, e teria pego o padrão que causou N-01 antes de

chegar a produção.

Não gaste esforço reescrevendo. A arquitetura segue correta, os motores centrais (BSM, GEX, RSI, MACD, breakeven) estão matematicamente verificados nesta auditoria por execução independente, e o time já demonstrou — nesta janela sem laudo formal — que consegue corrigir bem quando o problema está bem especificado. O que falta é fechar o último passo de cada correção: garantir que ela chegue a **todos** os consumidores do mesmo dado (não só o primeiro que alguém revisou) e que a tela pare de afirmar sobre si mesma algo que o código, uma linha abaixo, não sustenta.

Laudo elaborado por leitura direta do código-fonte no disco do usuário (via ponte remota), execução real de tsc, eslint e vitest no próprio ambiente do projeto, e reexecução independente dos motores BSM e GEX com casos de teste elaborados por este auditor e comparados contra cálculo manual em Python. Nenhum resultado deste laudo foi aceito por leitura de comentário no código sem verificação — onde não foi possível verificar por execução (ex.: A-11, A-13, A-17, A-23), isso está declarado explicitamente na Seção 3.1 como “não re-verificado”, e não deve ser lido como “resolvido”.